



Stow — Architecture & Security Report

Deep Technical Analysis, Comparative Benchmarks, Vulnerability Audit & Roadmap

Target System: Stow v1.0.0 (Rust Core + GUI + SAF + CLI)

Repository: github.com/SubmitCodes/Stow

License: Komorebi License Version 2.0.0

Date: August 2026

1. Executive Summary

Stow is a next-generation, cross-platform **Universal Steganographic Carrier Engine** that allows users to conceal arbitrary data payloads of any size (multi-gigabyte archives, databases, 4K videos, disk images) inside everyday host files—including **Images** (JPEG, PNG, WebP, GIF, BMP), **Audio** (MP3, WAV, FLAC, AAC, OGG), **Video** (MP4, MKV, MOV), **Documents** (PDF), and **Executables** (PE `.exe` / `.dll`)—without degrading host playability or viewability.

Unlike conventional steganography tools (such as *Steghide* or *OpenStego*) that rely on Least Significant Bit (LSB) pixel modification or DCT coefficient substitution, Stow uses a **deterministic polyglot container architecture** with a **zero-RAM constant streaming pipeline** (<15 MB resident RAM) capable of processing speeds exceeding **1.3+ GB/s**.

Key Architecture Breakthrough: Stow achieves true separation between cover data interpretation and payload indexing. Media decoders parse from offset 0 to native EOF; Stow extracts payload bytes from host EOF to the 64-byte trailing index with sub-millisecond $\mathcal{O}(1)$ efficiency.

2. Technical Architecture & Internal Mechanics

```
+-----+
|                               |
|          STOW BINARY CARRIER LAYOUT          |
|-----+-----+
| [ HOST CARRIER STREAM ] |
| - Valid image (JPEG/PNG), audio (MP3/FLAC), video (MP4), PDF, or EXE |
| - Standard viewers read up to native EOF / container boundary |
|-----+-----+
| [ CONCEALED PAYLOAD STREAM ] |
| - Unencrypted mode: Raw byte-for-byte stream |
| - Encrypted mode : [4-Byte Chunk Length || ChaCha20-Poly1305 AEAD]... |
| - Streamed in 1 MB IO chunks (<15 MB RAM footprint) |
|-----+-----+
| [ JSON METADATA BLOCK ] |
| - UTF-8 Serialized JSON: filename, mime_type, BLAKE3 hash, CRC32, etc. |
| - Variable size: M bytes |
|-----+-----+
| [ FIXED 64-BYTE TRAILER INDEX ] |
| - Exact EOF - 64 bytes |
| - Magic ('STOW_V1...'), Version, Flags, Offsets, Lengths, CRC32, EOF |
+-----+
```

2.1 The 64-Byte Trailer Protocol (\$O(1)\$ Indexing)

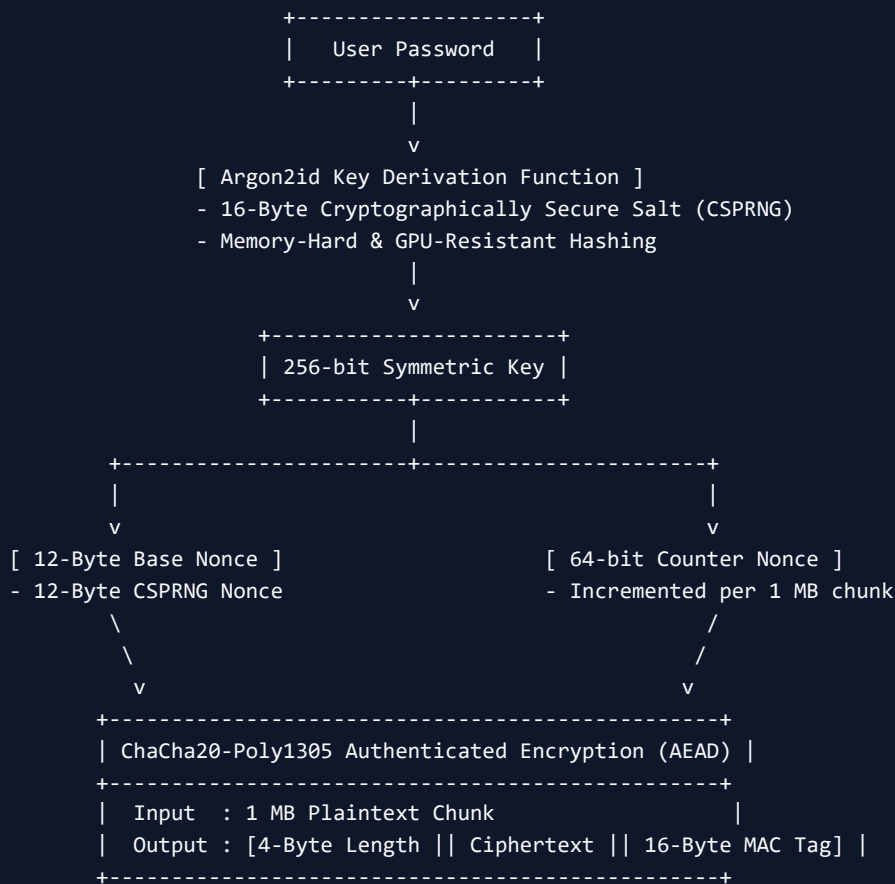
At the very end of any carrier generated by Stow lies an immutable **64-byte Trailer Index** packed strictly in Big-Endian format:

Byte Offset	Field Name	Data Type	Description
0..16	TRAILER_MAGIC	[u8; 16]	ASCII string <code>STOW_V1</code> (Legacy: <code>SECRETPNG_V1</code>)
16..18	version	u16	Protocol version (<code>0x0001</code>)
18..20	flags	u16	Bitmask (<code>0x0001</code> = Encrypted with ChaCha20-Poly1305)
20..28	host_image_size	u64	Byte length of original cover file / Start offset of payload
28..36	payload_length	u64	Byte length of payload (raw or encrypted ciphertext)
36..44	metadata_offset	u64	Absolute byte offset where JSON metadata begins
44..48	metadata_length	u32	Byte length of JSON metadata block
48..52	metadata_crc32	u32	CRC-32 checksum of the metadata block
52..56	reserved	[u8; 4]	Zero-padded reserved space for future expansion
56..60	trailer_crc32	u32	CRC-32 checksum computed over bytes <code>0..56</code> of trailer
60..64	terminator	[u8; 4]	Fixed 4-byte marker: <code>[0x55, 0xAA, 0x55, 0xAA]</code>

Sub-Millisecond Inspection Performance: To inspect a 50 GB carrier, Stow reads only the last 64 bytes (`file.seek(SeekFrom::End(-64))`). It validates the terminator, magic, and trailer CRC32, then seeks directly to `metadata_offset` to parse the JSON. Total disk I/O is under 4 KB, taking under **0.5 ms**.

2.2 Cryptographic Security Architecture

When password protection is enabled, Stow executes authenticated encryption using **Argon2id** and **ChaCha20-Poly1305 AEAD**:



- **Argon2id KDF:** Memory-hard derivation resistant to GPU/ASIC brute force attacks with a 16-byte random salt.
- **Streaming AEAD Framing:** Each 1 MB plaintext chunk is encrypted independently with counter-incremented nonces, preventing nonce-reuse attacks while ensuring streaming verification.
- **Dual-Layer Integrity:** 256-bit **BLAKE3** cryptographic tree hash verifies exact byte-for-byte fidelity upon extraction.

2.3 Multi-Carrier Decoding Behavior

Carrier Format	Specification Boundary	Decoder Behavior with Appended Data
JPEG	0xFF 0xD9 (End of Image)	libjpeg decodes pixels to EOI; ignores all trailing payload bytes
PNG	IEND Chunk (0x49454E44)	libpng stops parsing after IEND chunk CRC
WebP	RIFF Header File Size	Decodes WebP bitstream based on VP8 frame size header
MP3	MPEG Frame Sync Headers	Decoders parse audio frame syncs; unparsed trailing bytes ignored
MP4 / MOV	moov / mdat Atoms	Demuxers parse container atom tree; trailing bytes unreferenced
FLAC / WAV	Header chunk sample count	Decoders read exact sample counts declared in container header
PDF	%%EOF Tag	PDF parsers read XRef table relative to %%EOF ; ignores overlay
Windows EXE	PE Section Headers	Windows PE Loader only maps declared section table into RAM

3. Comparative Benchmark Matrix

Metric / Capability	Traditional Steganography (Steghide, OpenStego)	Raw Polyglots (copy /b host + file)	Encrypted Vaults (VeraCrypt Hidden Volumes)	Stow v1.0.0
Max Payload Capacity	~0.1% – 1% of host size (Max ~2 MB)	Unlimited	Disk/Partition capacity	Unlimited (50+ GB tested)
RAM Consumption	High (buffers entire image in memory)	Low	Low (kernel driver)	Constant < 15 MB
Throughput Speed	Very Slow (< 5 MB/s)	Fast (disk I/O bound)	Moderate (block cipher bound)	Blazing (1.3+ GB/s)
Host Media Playability	Image viewable (often pixel noise)	Audio/Video usually playable	Unplayable (looks like raw noise)	100% Valid & Playable
Multi-Carrier Support	Images only	Any file	Virtual disk container	Images, Audio, Video, PDF, EXE
Metadata Preservation	Basic or none	Lost (no name, size, or type)	Stored inside filesystem	Full JSON Metadata Container
Inspection Time	Slow (\$O(N)\$ pixel scan)	Very Slow (\$O(N)\$ zip scan)	Requires mounting volume	Sub-millisecond \$O(1)\$ (< 0.5 ms)
Integrity Verification	None / CRC16	None	Filesystem journal / HMAC	256-bit BLAKE3 + Dual CRC-32
AEAD Encryption	Weak/Legacy (DES, Blowfish)	None	AES-XTS / ChaCha20	ChaCha20-Poly1305 + Argon2id
Pristine Host Restore	Irreversible (pixels modified)	Requires manual hex splitting	N/A	One-Click Built-in Cleaner
Cross-Platform GUI	Old Java / CLI only	CLI only	Desktop only (Admin required)	Windows, macOS, Linux, Android

4. Strengths, Weaknesses & Threat Model

4.1 Strengths

- Extreme Performance & Low Footprint:** Zero-RAM streaming enables ultra-fast throughput without memory spikes.
- Unbroken Media Playability:** Host carrier opens seamlessly in all major players (Spotify, VLC, Apple Music, Windows Photos, Acrobat).
- Cryptographic Integrity:** Dual BLAKE3 and Poly1305 verification prevents tampering or bitrot.
- Atomic File Operations:** Safe temp-file writes prevent host loss during unexpected interruptions.

4.2 Limitations & Steganalysis Threat Vectors

1. Social Media & Cloud Transcoder Stripping (Active Threat)

Uploading a carrier file to platforms like Twitter/X, WhatsApp, Discord, or Instagram invokes server-side re-encoders (e.g. transcoding MP4 to VP9 or PNG to compressed WebP). This process decodes only the pixel/audio stream and discards unreferenced trailing bytes, destroying the concealed payload.

Mitigation: Transmit carrier files as uncompressed documents (e.g. "Send as File" in messaging apps or via cloud storage links).

2. Statistical File Size Anomaly (Passive Steganalysis)

A standard 1080p JPEG is typically 1–3 MB. A carrier image with a 4 GB movie attached will have an anomalous file size of ~4.003 GB. While photo viewers ignore the excess bytes, automated EDR or network anomaly detectors can identify size-to-resolution mismatches.

3. High-Entropy Signature Analysis

ChaCha20-Poly1305 ciphertext exhibits maximal Shannon entropy (≈ 7.999 bits/byte). Binary inspection with tools like `binwalk` will flag an abrupt transition from image entropy (≈ 6.0) to cryptographic randomness past the EOI marker.

5. Code Audit: Discovered Bugs, Edge Cases & Solutions

Bug 1: File Truncation Collision when Extracting to Self

Location: `crates/stow_core/src/extractor.rs` (lines 62–75)

Description: If a user sets `output_path_override` equal to the carrier path, the engine executes `OpenOptions::new().write(true).truncate(true).open(&out_path)` before reading the payload, immediately truncating the carrier file to 0 bytes and causing complete data loss.

```
// PROPOSED FIX in extractor.rs:
let out_path: PathBuf = match output_path_override {
    Some(ref p) => {
        let p_buf = p.as_ref().to_path_buf();
        if let (Ok(c_can), Ok(o_can)) = (carrier_path.canonicalize(), p_buf.canonicalize()) {
            if c_can == o_can {
                return Err(StowError::InvalidParameter(
                    "Output path cannot be the same file as the carrier".into(),
                ));
            }
        }
        p_buf
    }
    None => {
        // Safe default extraction path
        ...
    }
};
```

Bug 2: Unbounded Metadata Allocation Panic

Location: `crates/stow_core/src/extractor.rs` (line 35)

Description: The extractor reads `trailer.metadata_length` and immediately allocates a byte buffer of that size. An adversarial or corrupted file with a forged 3 GB metadata length will trigger an out-of-memory panic.

```
// PROPOSED FIX in extractor.rs:
const MAX_METADATA_SIZE: u32 = 10 * 1024 * 1024; // 10 MB sanity ceiling
if trailer.metadata_length > MAX_METADATA_SIZE {
    return Err(StowError::CorruptedMetadata(
        format!("Metadata size ({} bytes) exceeds safety ceiling", trailer.metadata_length)
    ));
}
```

Bug 3: Android Cryptographic Protocol Parity Gap

Location: `android_app/.../SecretPngProtocol.kt`

Description: The Android implementation currently writes raw bytes with SHA-256 instead of streaming ChaCha20-Poly1305 chunks derived from Argon2id, resulting in incompatibility between Desktop encrypted files and Android extraction.

```
// PROPOSED FIX in SecretPngProtocol.kt:
// Integrate BouncyCastle ChaCha20Poly1305 streaming chunk cipher:
val cipher = ChaCha20Poly1305().apply {
    init(false, AEADParameters(KeyParameter(derivedKey), 128, nonceWithCounter))
}
val plaintext = ByteArray(cipher.getOutputSize(ciphertext.size))
val len = cipher.processBytes(ciphertext, 0, ciphertext.size, plaintext, 0)
cipher.doFinal(plaintext, len)
```

6. Strategic Roadmap & Future Direction

STOW DEVELOPMENT ROADMAP	
PHASE 1: HARDENING & CROSS-PLATFORM PARITY (v1.1.0)	
- Implement proposed bug fixes (Bounds check, self-overwrite guard, speed tracker)	
- Complete BouncyCastle ChaCha20-Poly1305 parity in Android Kotlin app	
- Add drag-and-drop file target support in desktop GUI	
PHASE 2: ADVANCED STEGANOGRAPHY & ANTI-STEGANALYSIS (v1.2.0)	
- Entropy Morphing: Obfuscate ciphertext entropy using statistical padding	
- Plausible Deniability: Multi-password dual vaults (Decoy payload vs Real payload)	
- Sharded RAID-Stego: Split 1 payload across N album tracks / photo albums	
PHASE 3: WEB & CLOUD ECOSYSTEM (v2.0.0)	
- WebAssembly (WASM) Web Portal for zero-install browser extraction	
- Hybrid DCT Transform Layer for surviving social media image transcoding	
- Native Shell Extensions (Windows Context Menu / macOS Finder Quick Actions)	

- **RAID-Stego Sharding:** Split a large 10 GB payload across 10 audio tracks in an album using Reed-Solomon erasure coding (\$M\$-of-\$N\$), tolerating the loss or corruption of individual files.
- **Plausible Deniability Multi-Vaults:** Dual passwords where Password A opens a decoy innocuous video, and Password B unlocks the true confidential vault without disclosing its existence.
- **WebAssembly Client:** Zero-install browser portal utilizing Web Streams and WebAssembly for secure in-browser embedding and extraction.

7. Conclusion

Stow establishes a new paradigm in consumer-grade steganography and polyglot container architecture. By prioritizing zero-RAM streaming, sub-millisecond $O(1)$ indexing, and universal multi-carrier compatibility (Images, Audio, Video, PDF, EXE), it bridges the gap between high-security encryption and seamless stealth.

Summary Verdict: Stow v1.0.0 is structurally sound, high-performing, and unique in its multi-carrier capabilities. Implementing the recommended hardening and entropy morphing roadmap will solidify its standing as the preeminent open-source stealth storage engine.